

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Trabajo Práctico N.º 5 Procesamiento y Optimización de Consultas

Alumno: Tomás Rodeghiero
Año: 2026

Índice

1. Introducción	2
2. Ejercicio 1: Algoritmos de selección	2
3. Ejercicio 2: Árbol de query y expresiones equivalentes	5
3.1. a) $\sigma_{desc="Pan Rallado" \wedge precio_venta < 15}(proveedor \bowtie provee \bowtie articulos)$	5
3.2. b) $\sigma_{precio > 25}(cliente \bowtie compran \bowtie articulos)$	6
4. Ejercicio 3: Estimación de tamaño y orden de join	8
5. Ejercicio 4: Catálogo de PostgreSQL	10
6. Ejercicio 5: Ejecución y planes en PostgreSQL	13
Conclusión	17

1. Introducción

El Práctico 5 cierra la parte del cuatrimestre dedicada a entender qué hace un motor de base de datos por dentro cuando recibe una consulta. Los cinco ejercicios cubren los tres bloques en los que se descompone el procesamiento de un *query*:

- **Procesamiento físico:** qué algoritmo concreto se usa para cada operación del álgebra relacional (Ejercicio 1).
- **Optimización lógica:** cómo se reescribe el árbol de la consulta usando reglas de equivalencia para bajar selecciones, ordenar joins y reducir tamaños intermedios (Ejercicios 2 y 3).
- **Catálogo y estadísticas:** dónde se guarda en PostgreSQL la información que el *planner* usa para decidir el plan, y cómo se influye sobre el plan elegido (Ejercicios 4 y 5).

Cada ejercicio sigue siempre la misma estructura: **Consigna** (texto literal del PDF), **Idea** (lectura natural y concepto detrás), **Solución** (algoritmo, árbol algebraico, fórmula o SQL según corresponda), **Por qué funciona / por qué es mejor** (lógica de la transformación o de la elección) y **En MySQL 8+** (consulta equivalente y/o herramientas para inspeccionar el plan: EXPLAIN ANALYZE, EXPLAIN FORMAT=JSON, optimizer_trace, hints).

Notación. A lo largo del documento se usa la notación clásica del álgebra relacional: σ para selección, Π para proyección y $\bowtie$ para join (natural cuando no se aclara la condición). Para los costos se siguen las variables del Teórico 6: t_T tiempo de transferencia de un bloque, t_b tiempo de búsqueda, b_r bloques de la relación r , n_r tuplas de r , h_i altura del índice.

2. Ejercicio 1: Algoritmos de selección

Consigna

Dado la siguiente tabla de una base de datos:

`curso(codigo, nombre, cantidad_inscriptos, cupo_maximo)`

Donde: `codigo` es clave primaria. Hay definido un índice secundario sobre el atributo `cantidad_inscriptos`. Hay definido un índice secundario sobre el atributo `cupo_maximo`.

Especifique qué algoritmos utilizaría (justifique) para las siguientes consultas:

- $\sigma_{codigo=1001}(curso)$
- $\sigma_{codigo<1000}(curso)$
- $\sigma_{cantidad_inscriptos=40}(curso)$
- $\sigma_{cantidad_inscriptos<50}(curso)$
- $\sigma_{cantidad_inscriptos=50 \wedge cupo_maximo>40}(curso)$

Idea

Se pide elegir, para cada selección, el algoritmo físico más eficiente entre los presentados en el Teórico 6 (A1–A11). La decisión depende de tres factores: si hay índice sobre el atributo del predicado, si la condición es de igualdad o de rango, y si hay varios predicados conjuntivos sobre atributos con índices distintos (caso de intersección de RIDs).

Solución

Los algoritmos relevantes del Teórico 6 son:

- **A1:** búsqueda lineal. Aplicable siempre. Costo $b_r \cdot t_T + t_b$; si la condición es por clave y hay igualdad, corta apenas encuentra: $(b_r/2) \cdot t_T + t_b$.
- **A2:** búsqueda binaria sobre archivo ordenado.
- **A3:** igualdad sobre índice de clave primaria o único. A lo sumo una tupla. Costo $(h_i + 1)(t_T + t_b)$.
- **A4:** igualdad sobre índice primario no único.
- **A5:** igualdad sobre índice secundario. Puede devolver varios registros, cada uno en bloque distinto.
- **A6:** rango sobre índice primario (recorre secuencialmente).
- **A7:** rango sobre índice secundario. Por cada puntero, una I/O.
- **A8:** conjunción usando *un* índice + chequeo en memoria.
- **A9:** conjunción usando un índice multiclave.
- **A10:** conjunción por intersección de identificadores (RIDs/punteros).
- **A11:** disyunción por unión de identificadores.

Resolución por inciso:

a) $\sigma_{\text{codigo}=1001}(\text{curso})$: **A3** (igualdad sobre índice primario/único). **codigo** es PK y, por convención del curso, toda PK lleva un índice único (B+tree). La condición es igualdad sobre ese atributo; A3 recupera a lo sumo una tupla con costo $(h_i + 1)(t_T + t_b)$.

b) $\sigma_{\text{codigo}<1000}(\text{curso})$: **A6** (rango sobre índice primario). Misma convención de PK con índice. Si se interpreta la PK sin índice primario explícito, queda **A1**; en cualquier caso una solución con índice (B+tree) es preferible siempre que la selectividad no sea muy alta.

c) $\sigma_{\text{cantidad_inscriptos}=40}(\text{curso})$: **A5** (igualdad sobre índice secundario). Hay índice secundario sobre el atributo y la condición es de igualdad. Como **cantidad_inscriptos** no es clave, pueden satisfacer la condición varios registros, posiblemente uno por bloque. Costo peor caso $(h_i + n)(t_T + t_b)$ con n = tuplas matcheadas.

d) $\sigma_{\text{cantidad_inscriptos}<50}(\text{curso})$: **A7** (rango sobre índice secundario). Recorre las hojas del índice secundario para conseguir los punteros que cumplen < 50 y, por cada puntero, hace una I/O. Atractivo cuando la selectividad es baja; si la mayoría cumple, un *Seq Scan* (A1) puede ser preferible.

e) $\sigma_{\text{cantidad_inscriptos}=50 \wedge \text{cupo_maximo}>40}(\text{curso})$: **A10** (intersección de identificadores). Hay índice secundario para cada condición:

- **cantidad_inscriptos = 50** con el índice de **cantidad_inscriptos** (lógica A5) devuelve P_1 .
- **cupo_maximo > 40** con el índice de **cupo_maximo** (lógica A7) devuelve P_2 .
- Se calcula $P_1 \cap P_2$ y se recuperan sólo las tuplas que sobreviven.

Alternativa: A8 (elegir la condición más selectiva, recuperar por su índice, testear el resto en memoria); razonable si una condición es claramente más selectiva.

Tabla resumen:

Inciso	Condición	Algoritmo
a	<code>codigo = 1001</code>	A3
b	<code>codigo < 1000</code>	A6
c	<code>cantidad_inscriptos = 40</code>	A5
d	<code>cantidad_inscriptos < 50</code>	A7
e	<code>cantidad_inscriptos = 50 ∧ cupo_maximo > 40</code>	A10

Por qué funciona / por qué es mejor

- **Igualdad por PK (a):** con un índice único, h_i típicamente 3–4 bloques en B+tree; comparado contra b_r páginas de un *Seq Scan*, A3 es órdenes de magnitud más barato.
- **Rango por PK (b):** A6 aprovecha que el archivo está ordenado (o que el índice primario indica el orden físico) para recorrer secuencialmente los bloques que matchean.
- **Índice secundario (c, d):** el costo escala con el número de tuplas matcheadas (cada una potencialmente en un bloque distinto). Si la selectividad es muy alta, A1 puede ganar porque evita los saltos.
- **Intersección de RIDs (e):** combinar dos índices vía intersección permite recuperar sólo los bloques que cumplen *ambas* condiciones, en vez de leer toda la tabla y filtrar. Si una condición es muy selectiva, A8 puede ser preferible por simplicidad.

En MySQL 8+

MySQL 8+ usa nombres distintos para los mismos algoritmos. `EXPLAIN` indica el algoritmo elegido en la columna `type` y/o `Extra`:

Teórico	MySQL <code>EXPLAIN.type</code>	Observación
A1	ALL	Full table scan.
A3	const / eq_ref	Igualdad por PK/UNIQUE.
A5	ref	Igualdad por índice secundario.
A6	range	Rango usando PRIMARY.
A7	range	Rango usando índice secundario.
A10	index_merge	+ Extra: Using intersect(...)
A11	index_merge	+ Extra: Using union(...)

```
-- Verificacion en MySQL (asumiendo practico5_mysql.curso con sus indices)
EXPLAIN SELECT * FROM curso WHERE codigo = 1001;
-- type=const / key=PRIMARY -> A3

EXPLAIN SELECT * FROM curso WHERE codigo < 1000;
-- type=range / key=PRIMARY -> A6

EXPLAIN SELECT * FROM curso WHERE cantidad_inscriptos = 40;
-- type=ref / key=idx_curso_cantidad -> A5

EXPLAIN SELECT * FROM curso WHERE cantidad_inscriptos < 50;
-- type=range / key=idx_curso_cantidad -> A7

EXPLAIN SELECT * FROM curso
WHERE cantidad_inscriptos = 50 AND cupo_maximo > 40;
-- type=index_merge / Extra: Using intersect(idx_curso_cantidad, idx_curso_cupo);
-- Using where -> A10
```

Diferencias prácticas con PostgreSQL/Oracle:

- En MySQL/InnoDB, la tabla está *clustered* por la PK: A3 y A6 no requieren un acceso extra a la tabla porque el índice y los datos viven en la misma estructura.
- Si MySQL no detecta el *index_merge* y elige sólo uno de los índices, se puede forzar con la sintaxis `... FROM curso USE INDEX FOR ORDER BY (...)` o, más directamente, con la variable `optimizer_switch` (`SET optimizer_switch='index_merge=on,index_merge_intersection=on'`).

3. Ejercicio 2: Árbol de query y expresiones equivalentes

Consigna

Dado el siguiente esquema de base de datos:

- `articulos(Nart, desc, precio, cant, stock_min, stock_max)`
- `provee(Nprov, Nart, precio_venta)`
- `proveedor(Nprov, nombre, direccion)`
- `cliente(Ncli, nombre, direccion)`
- `compran(Ncli, Nart)`

Obtenga el árbol de query y tres expresiones equivalentes (justifique cada paso de la transformación) para:

- $\sigma_{descripcion="Pan Rallado" \wedge precio_venta < 15}(proveedor \bowtie provee \bowtie articulos)$
- $\sigma_{precio > 25}(cliente \bowtie compran \bowtie articulos)$

Idea

El ejercicio pide aplicar las *reglas de equivalencia* del Teórico 7 para reescribir cada consulta en un árbol más eficiente. Las heurísticas centrales son: empujar selecciones hacia las hojas (achican cardinalidades antes de los joins), y reordenar los joins para que los resultados intermedios sean lo más pequeños posible.

Solución

Reglas relevantes del Teórico 7:

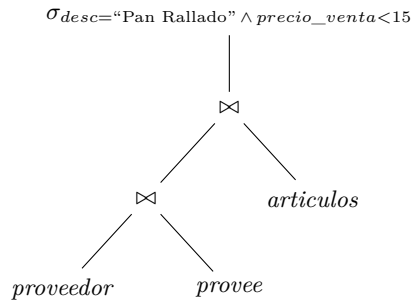
- **R1** (cascada): $\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$.
- **R2** (conmutatividad de σ): $\sigma_{\theta_1}(\sigma_{\theta_2}(E)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(E))$.
- **R5** (conmutatividad de $\bowtie$): $E_1 \bowtie E_2 \equiv E_2 \bowtie E_1$.
- **R6a** (asociatividad): $(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$.
- **R7a** (push-down sobre un operando): $\sigma_{\theta_0}(E_1 \bowtie E_2) \equiv \sigma_{\theta_0}(E_1) \bowtie E_2$ si θ_0 refiere sólo a E_1 .
- **R7b** (push-down distributivo): $\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie E_2) \equiv \sigma_{\theta_1}(E_1) \bowtie \sigma_{\theta_2}(E_2)$ si cada θ_i refiere sólo a E_i .

3.1. a) $\sigma_{desc="Pan Rallado" \wedge precio_venta < 15}(proveedor \bowtie provee \bowtie articulos)$

Expresión base E_0 :

$$E_0 = \sigma_{desc="Pan Rallado" \wedge precio_venta < 15} (proveedor \bowtie provee \bowtie articulos)$$

Árbol de E_0 :



Equivalente E_1 — cascada de selecciones ($R1$):

$$E_1 = \sigma_{desc="Pan Rallado"}(\sigma_{precio_venta < 15}((proveedor \bowtie provee) \bowtie articulos))$$

Reemplaza la selección conjuntiva por dos encadenadas. No mejora costo todavía, pero habilita el push-down: cada σ tiene un único predicado.

Equivalente E_2 — push-down por $R7b$:

Atributos: **precio_venta** sólo en **provee**; **desc** sólo en **articulos**. Aplicando $R7a/R7b$ cada selección baja a la rama correspondiente:

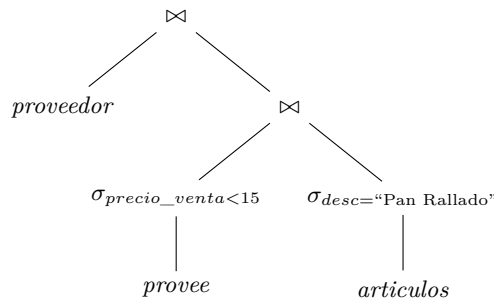
$$E_2 = (proveedor \bowtie \sigma_{precio_venta < 15}(provee)) \bowtie \sigma_{desc="Pan Rallado"}(articulos)$$

Equivalente E_3 — reasociación ($R5$, $R6a$):

Conviene hacer primero el join entre las dos relaciones *ya filtradas* (que son las más chicas):

$$E_3 = proveedor \bowtie (\sigma_{precio_venta < 15}(provee) \bowtie \sigma_{desc="Pan Rallado"}(articulos))$$

Árbol final (E_3):

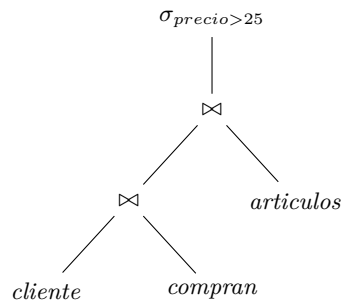


3.2. b) $\sigma_{precio > 25}(cliente \bowtie compran \bowtie articulos)$

Expresión base E_0 :

$$E_0 = \sigma_{precio > 25}((cliente \bowtie compran) \bowtie articulos)$$

Árbol de E_0 :



Equivalente E_1 — *push-down a **articulos** (R7a)*: **precio** sólo aparece en **articulos**.

$$E_1 = (\text{cliente} \bowtie \text{compran}) \bowtie \sigma_{\text{precio} > 25}(\text{articulos})$$

Equivalente E_2 — *asociatividad (R6a)*:

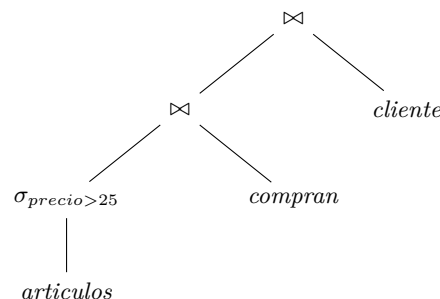
$$E_2 = \text{cliente} \bowtie (\text{compran} \bowtie \sigma_{\text{precio} > 25}(\text{articulos}))$$

Equivalente E_3 — *conmutatividad y reasociación (R5, R6a)*:

$$E_3 = (\sigma_{\text{precio} > 25}(\text{articulos}) \bowtie \text{compran}) \bowtie \text{cliente}$$

Equivalente operacionalmente a E_2 pero útil si el algoritmo de join es asimétrico (ej. *hash join* con la relación más chica como *build*).

Árbol final (E_3):



Por qué funciona / por qué es mejor

- **Cascada (R1)** no mejora por sí sola: solo descompone la selección conjuntiva en dos selecciones independientes, que recién después se pueden mover por separado.
- **Push-down (R7a/R7b)** es la transformación que más reduce costo: las selecciones se evalúan sobre tablas base y la cardinalidad cae *antes* de los joins, así los joins trabajan sobre relaciones más chicas.
- **Reasociación (R6a)** elige el orden de joins que minimiza el tamaño del resultado intermedio. Hacer primero el join entre las dos relaciones ya filtradas suele dar el intermedio más pequeño.
- Si no se aplicaran estas reglas, el motor primero haría los joins completos (potencialmente con millones de tuplas intermedias) y filtraría al final, multiplicando el costo de I/O.

En MySQL 8+

El optimizador de MySQL 8+ aplica internamente estas mismas transformaciones (predicado push-down, join reordering). Para *ver* qué decisiones tomó, hay dos herramientas:

```
-- 1) EXPLAIN FORMAT=JSON da el plan estructurado con costos y filas estimadas.
EXPLAIN FORMAT=JSON
SELECT *
FROM proveedor pv
JOIN provee pr ON pv.Nprov = pr.Nprov
JOIN articulos a ON pr.Nart = a.Nart
WHERE a.desc = 'Pan Rallado' AND pr.precio_venta < 15;

-- 2) optimizer_trace muestra cada transformacion considerada
SET optimizer_trace = "enabled=on";
SELECT /* la query del Ejercicio 2.a */ ...;
SELECT TRACE
FROM information_schema.OPTIMIZER_TRACE;
SET optimizer_trace = "enabled=off";
```

En la TRACE aparecen pasos como `condition_processing` (donde se ve el push-down del WHERE sobre cada tabla), `rows_estimation` (cardinalidades estimadas) y `considered_execution_plans` (cada orden de join evaluado con su costo).

Diferencias relevantes con PostgreSQL/Oracle:

- MySQL usa principalmente *block nested loop* históricamente; *hash join* fue introducido en 8.0.18 (octubre 2019) y, desde 8.0.20, se aplica también a equi-joins con índices. *Sort-merge* no existe como algoritmo separado.
- Para forzar el orden de join en MySQL: `STRAIGHT_JOIN` (a nivel sentencia o entre tablas) bloquea el reordenamiento que haría el optimizador.

4. Ejercicio 3: Estimación de tamaño y orden de join

Consigna

Dadas las relaciones $R_1(A, B, C)$, $R_2(C, D, E)$, $R_3(E, F)$ con claves A , C y E respectivamente. Suponer que r_1 tiene 250 tuplas, r_2 3000 tuplas y r_3 5000 tuplas. Estimar el tamaño de $r_1 \bowtie r_2 \bowtie r_3$ y dar el orden de ejecución más eficiente para calcular el join.

Idea

Cuando dos relaciones se unen por un atributo que es *clave* en una de ellas, cada tupla de la otra cruza con a lo sumo una; la cardinalidad del join queda acotada por la relación no-clave. Eso da una regla directa para acotar tamaños intermedios y, por lo tanto, elegir el orden de ejecución del join multirrelación.

Solución

Marco teórico (Teórico 7):

Si $R \cap S = \{A\}$ y A es clave de S , entonces $|R \bowtie S| \leq |R|$. Si A no es clave en ninguna, la estimación clásica es $|R \bowtie S| \approx n_r n_s / \max\{V(A, r), V(A, s)\}$. La heurística para ordenar joins es: primero los joins que reduzcan más el resultado intermedio.

Paso 1: $R_1 \bowtie R_2$ (atributo común C): C es clave en R_2 , así que cada tupla de R_1 matchea con a lo sumo una de R_2 :

$$|R_1 \bowtie R_2| \leq |R_1| = 250.$$

Asumiendo integridad referencial usual (cada $R_1.C$ está presente en $R_2.C$), se cumple la igualdad: $|R_1 \bowtie R_2| = 250$.

Paso 2: $(R_1 \bowtie R_2) \bowtie R_3$ (atributo común E): E es clave en R_3 , por lo tanto cada tupla intermedia matchea con a lo sumo una de R_3 :

$$|R_1 \bowtie R_2 \bowtie R_3| \leq |R_1 \bowtie R_2| = 250.$$

Resultado: la cardinalidad estimada es $|R_1 \bowtie R_2 \bowtie R_3| = 250$ tuplas.

Orden de ejecución — dos planes razonables:

Plan A: $(R_1 \bowtie R_2) \bowtie R_3$.

- Intermedio: $|R_1 \bowtie R_2| = 250$ tuplas.
- Join final: 250 tuplas contra R_3 (5000 tuplas) por E (clave en R_3).

Plan B: $R_1 \bowtie (R_2 \bowtie R_3)$.

- Intermedio: $|R_2 \bowtie R_3|$. Como E es clave en R_3 , $|R_2 \bowtie R_3| \leq |R_2| = 3000$. Bajo integridad referencial, $|R_2 \bowtie R_3| = 3000$.
- Join final: 3000 tuplas contra R_1 (250 tuplas) por C (clave en R_1).

Plan	Tamaño intermedio	Tamaño final
A: $(R_1 \bowtie R_2) \bowtie R_3$	250	250
B: $R_1 \bowtie (R_2 \bowtie R_3)$	3000	250

Orden recomendado: $((R_1 \bowtie R_2) \bowtie R_3)$ con tamaño estimado 250.

Por qué funciona / por qué es mejor

- El Plan A da un intermedio 12 veces más chico que el Plan B. Ese resultado se mantiene en pipeline o se materializa, y se compara contra R_3 en el último join.
- Cualquier algoritmo concreto (*nested loop*, *hash join*, *merge join*) trabaja mejor con la relación externa más pequeña; el Plan A se la garantiza.
- Si las claves no estuvieran indicadas, la estimación bajaría a la fórmula con $V(A, r)$ y el optimizador necesitaría las estadísticas del catálogo para elegir bien.

En MySQL 8+

MySQL 8+ aplica internamente la misma heurística de reordenamiento (*table pull-out*, *nested-loop search*). Para *verificar* el orden elegido:

```
EXPLAIN FORMAT=JSON
SELECT *
FROM R1 JOIN R2 ON R1.C = R2.C
      JOIN R3 ON R2.E = R3.E;
-- En el JSON aparece "nested_loop" o "hash_join" con el orden:
-- primer table: R1 (filas estimadas: 250)
-- segunda:      R2 (filas filtradas tras el join)
-- tercera:      R3
```

Para *forzar* un orden distinto y comparar costos:

```
-- STRAIGHT_JOIN bloquea el reordenamiento del optimizador:
SELECT STRAIGHT_JOIN *
FROM R2 JOIN R3 ON R2.E = R3.E      -- fuerza Plan B
```

```

JOIN R1 ON R1.C = R2.C;

-- Alternativa con hint (MySQL 8+):
SELECT /*+ JOIN_ORDER(R1, R2, R3) */ *
FROM R1, R2, R3
WHERE R1.C = R2.C AND R2.E = R3.E;

```

Diferencia clave con PostgreSQL/Oracle: en MySQL, la estimación de cardinalidad depende fuertemente del estadístico `rows` de `INFORMATION_SCHEMA.TABLES` y de los índices; si `ANALYZE TABLE` no se ejecutó hace tiempo, el optimizador puede elegir el Plan B por estadísticas obsoletas. PostgreSQL/Oracle tienen `autoanalyze` con umbrales más sofisticados; en MySQL conviene ejecutar `ANALYZE TABLE` explícito ante cargas grandes.

5. Ejercicio 4: Catálogo de PostgreSQL

Consigna

- Investigue el catálogo de una base de datos postgres. ¿En qué Tabla/s y/o Vista/s postgres almacena información sobre las tablas, atributos y sus tipos? Haga una consulta al catálogo de la base de datos del ejercicio 1 b) de la práctica 1 y obtenga nombre y tipo de los atributos de la tabla “vehículo”.
- Continúe con la investigación del catálogo de una base de datos postgres. ¿En qué Tabla/s y/o Vista/s postgres almacena información estadística de las tablas? ¿Qué tipo de estadística almacena para los atributos de las tablas?

Idea

PostgreSQL expone su metadata por dos caminos: las vistas estándar `information_schema.*` (portables, conformes al SQL estándar) y las tablas del catálogo interno `pg_catalog.*` (específicas de PostgreSQL, más completas). Las estadísticas que usa el planner viven principalmente en `pg_class` (a nivel tabla) y `pg_statistic/pg_stats` (a nivel columna).

Solución

a) *Metadata de tablas, atributos y tipos.*

Estándar `information_schema`:

- `information_schema.tables`: tablas y vistas accesibles.
- `information_schema.columns`: una fila por columna, con `column_name`, `data_type`, `udt_name`, longitud/precisión, nullability, default, posición.

Catálogo interno `pg_catalog`:

- `pg_class`: objetos relacionales (tablas, índices, secuencias, vistas).
- `pg_namespace`: esquemas.
- `pg_attribute`: columnas de cada relación.
- `pg_type`: catálogo de tipos.

Consulta pedida (nombre y tipo de los atributos de `vehículo`):

```

-- Version estandar
SELECT c.column_name AS atributo,
       c.data_type   AS tipo,
       c.udt_name    AS tipo_interno

```

```

FROM information_schema.columns c
WHERE c.table_schema = 'public'
      AND c.table_name = 'vehiculo'
ORDER BY c.ordinal_position;

-- Version pg_catalog (mas cercana al catalogo real)
SELECT a.attname AS atributo,
       format_type(a.atttypid, a.atttypmod) AS tipo
FROM pg_catalog.pg_attribute a
JOIN pg_catalog.pg_class c ON c.oid = a.attrelid
JOIN pg_catalog.pg_namespace n ON n.oid = c.relnamespace
WHERE n.nspname = 'public'
      AND c.relname = 'vehiculo'
      AND a.attnum > 0 -- excluye columnas del sistema
      AND NOT a.attisdropped -- excluye columnas borradas logicamente
ORDER BY a.attnum;

```

b) Estadísticas que usa el planner.

A nivel tabla — *pg_class*:

- **reltuples**: estimación de la cantidad de filas (actualizada por ANALYZE/VACUUM).
- **relpages**: cantidad estimada de páginas (bloques).

Las estadísticas de *actividad* (scans, inserts, último ANALYZE) están en *pg_stat_user_tables*.

```

SELECT n.nspname AS esquema,
       c.relname AS tabla,
       c.reltuples::bigint AS filas_estimadas,
       c.relpages AS paginas
FROM pg_catalog.pg_class c
JOIN pg_catalog.pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind = 'r'
      AND c.relname = 'vehiculo';

```

A nivel columna — *pg_stats* (vista sobre *pg_statistic*, decodificada):

- **null_frac**: fracción de valores NULL.
- **n_distinct**: cantidad de valores distintos (positivo, o negativo si se expresa como fracción).
- **avg_width**: ancho promedio en bytes.
- **most_common_vals** / **most_common_freqs**: MCV (Most Common Values) y sus frecuencias. Permiten estimar selectividad para igualdades sobre valores populares.
- **histogram_bounds**: histograma equi-depth para selectividad de rangos.
- **correlation**: correlación entre el orden lógico del atributo y el orden físico en disco. Cercano a 1 favorece Index Scan.

```

SELECT s.attname,
       s.null_frac,
       s.n_distinct,
       s.avg_width,
       s.correlation,
       s.most_common_vals,
       s.most_common_freqs,
       s.histogram_bounds
FROM pg_catalog.pg_stats s
WHERE s.schemaname = 'public'
      AND s.tablename = 'vehiculo'
ORDER BY s.attname;

```

Si `pg_stats` devuelve datos desactualizados o vacíos, hay que ejecutar `ANALYZE public.vehiculo;`. El recolector se dispara en `VACUUM ANALYZE` o cuando `autovacuum` considera que se acumularon suficientes cambios.

Por qué funciona / por qué es mejor

- `information_schema` es la fachada portable, pero `pg_catalog` expone columnas que no están en el estándar (como `oid`, `relkind`, `atttypid`).
- Los datos de `pg_stats` son los n_r , $V(A, r)$, MCVs e histogramas del Teórico 7. El planner los usa para estimar el tamaño de cada selección ($\sigma_{A=v}(r) \approx n_r/V(A, r)$), de los joins ($n_r n_s / \max(V(A, r), V(A, s))$) y para decidir entre Seq Scan / Index Scan / Bitmap Heap Scan.
- `correlation` influye en el costo estimado de un Index Scan: con correlación alta, el motor sabe que las lecturas serán mayormente secuenciales (más baratas).

En MySQL 8+

MySQL 8+ expone metadata estadística por tres caminos:

1) *INFORMATION_SCHEMA* (estándar, portable):

```
-- Equivalente a pg_class.reltuples / pg_class.relpages
SELECT TABLE_NAME, ENGINE, TABLE_ROWS, AVG_ROW_LENGTH, DATA_LENGTH, INDEX_LENGTH
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = 'practico1b_mysql'
  AND TABLE_NAME   = 'vehiculo';

-- Equivalente a pg_attribute / pg_type
SELECT COLUMN_NAME, DATA_TYPE, COLUMN_TYPE, IS_NULLABLE, COLUMN_KEY
FROM information_schema.COLUMNS
WHERE TABLE_SCHEMA = 'practico1b_mysql'
  AND TABLE_NAME   = 'vehiculo'
ORDER BY ORDINAL_POSITION;

-- Cardinalidad estimada por indice (similar a n_distinct de pg_stats)
SELECT INDEX_NAME, COLUMN_NAME, CARDINALITY, SEQ_IN_INDEX
FROM information_schema.STATISTICS
WHERE TABLE_SCHEMA = 'practico1b_mysql'
  AND TABLE_NAME   = 'vehiculo'
ORDER BY INDEX_NAME, SEQ_IN_INDEX;
```

2) *Tablas internas de InnoDB para estadísticas persistentes:*

```
-- Stats por tabla (filas, paginas, paginas de indice)
SELECT * FROM mysql.innodb_table_stats
WHERE database_name = 'practico1b_mysql'
  AND table_name    = 'vehiculo';

-- Stats por columna de cada indice (n_diff_pfx??, leaf pages, ...)
SELECT * FROM mysql.innodb_index_stats
WHERE database_name = 'practico1b_mysql'
  AND table_name    = 'vehiculo';
```

3) *Histogramas (MySQL 8.0+)*: a diferencia de PostgreSQL que arma histogramas en `ANALYZE`, MySQL exige crearlos a mano:

```
ANALYZE TABLE vehiculo
UPDATE HISTOGRAM ON saldoActual, modelo WITH 100 BUCKETS;
```

```
-- Inspeccion
SELECT * FROM information_schema.COLUMN_STATISTICS
WHERE SCHEMA_NAME = 'practico1b_mysql'
AND TABLE_NAME = 'vehiculo';
```

Diferencias con PostgreSQL:

- PostgreSQL guarda MCVs e histogramas por defecto en `pg_stats` para cada columna (lo arma `ANALYZE`). MySQL no construye histogramas por defecto: hay que pedirlos con `ANALYZE TABLE ... UPDATE HISTOGRAM`. Sin histograma, MySQL solo tiene cardinalidades por índice.
- En PostgreSQL la columna `correlation` relaciona orden lógico/físico; MySQL/InnoDB no expone esa métrica directamente.

6. Ejercicio 5: Ejecución y planes en PostgreSQL

Consigna

Dado el siguiente esquema de una base de datos:

- `persona(persona_id, activar_mail_personal, ..., path_foto, caracteristica_celular)`
- `recurso(recurso_id, descripcion, id, nombre, orden)`
- `seguimiento_acceso(seguimiento_acceso_id, fecha_yhora_entrada, fecha_yhora_salida, host, persona_id, recurso_id)`

Realice lo siguiente:

- Cree las tablas en una base de datos postgres (el script de creación se puede descargar de la plataforma Evelia) y cargue datos en la tabla `persona` y `recursos`.
- Cree un programa que, por cada persona inserte al menos 1 registro en la tabla `seguimiento_acceso` por cada recurso (producto cartesiano entre `persona` y `recurso` más fecha y hora de acceso).
- Observe y analice en el catálogo la información estadística de las tablas creadas en este ejercicio.
- Luego de ejecutar la siguiente consulta:

```
SELECT * FROM seguimiento_acceso WHERE "FECHA_YHORA_ENTRADA" = '2023-05-06'
```

- observe el plan que usó postgres con `EXPLAIN`; (b) qué podría hacer para mejorar la performance y cambiar el plan.
- Luego de ejecutar `SELECT * FROM seguimiento_acceso NATURAL JOIN persona`: (a) analice su plan de ejecución; (b) modifique la consulta para que el motor utilice Merge Join en lugar de Hash Join.

Idea

Los cinco incisos forman un único hilo: cargar datos, mirar las estadísticas que el planner usa, observar planes con `EXPLAIN ANALYZE`, e influir sobre la elección del plan creando índices, ejecutando `ANALYZE` o usando `enable_hashjoin`. Es un *ejercicio de campo* sobre los conceptos de los Teóricos 6 y 7.

Solución

a) Creación de tablas y carga inicial.

```
\i practico-05/ejercicio-05/sql/PERSONAPostgres.sql
\i practico-05/ejercicio-05/sql/RecursoPostgres.sql
\i practico-05/ejercicio-05/sql/Seguimiento_AceesoPostgres.sql

SELECT 'persona'          AS tabla, count(*) FROM persona
UNION ALL
SELECT 'recurso',          count(*) FROM recurso
UNION ALL
SELECT 'seguimiento_acceso', count(*) FROM seguimiento_acceso;
```

b) Programa: una fila por par persona-recurso.

```
WITH pares AS (
    SELECT p.persona_id,
           r."RECURSO_ID" AS recurso_id,
           row_number() OVER (ORDER BY p.persona_id, r."RECURSO_ID") AS rn
    FROM persona p
    CROSS JOIN recurso r
),
faltantes AS (
    SELECT pa.*
    FROM pares pa
    WHERE NOT EXISTS (
        SELECT 1
        FROM seguimiento_acceso s
        WHERE s.persona_id = pa.persona_id
              AND s.recurso_id = pa.recurso_id
    )
)
INSERT INTO seguimiento_acceso
    (fecha_yhora_entrada, fecha_yhora_salida, host, persona_id, recurso_id)
SELECT TIMESTAMP '2023-05-01 00:00:00'
       + ((rn % 30) * INTERVAL '1 day')
       + ((rn % 86400) * INTERVAL '1 second'),
       TIMESTAMP '2023-05-01 00:20:00'
       + ((rn % 30) * INTERVAL '1 day')
       + ((rn % 86400) * INTERVAL '1 second'),
       'host-' || ((rn % 10) + 1),
       persona_id, recurso_id
FROM faltantes;
```

c) Estadísticas del catálogo.

```
ANALYZE persona;
ANALYZE recurso;
ANALYZE seguimiento_acceso;

-- Estimaciones del planner por tabla
SELECT n.nspname AS esquema, c.relname AS tabla,
       c.reltuples::bigint AS filas_estimadas, c.relpages AS paginas
FROM pg_catalog.pg_class c
JOIN pg_catalog.pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind = 'r'
      AND c.relname IN ('persona', 'recurso', 'seguimiento_acceso');

-- Actividad
SELECT schemaname, relname, seq_scan, idx_scan,
       n_tup_ins, n_live_tup, last_analyze
FROM pg_stat_user_tables
WHERE relname IN ('persona', 'recurso', 'seguimiento_acceso');
```

```
-- Por columna
SELECT schemaname, tablename, attname, null_frac, n_distinct, avg_width,
       correlation, most_common_vals, histogram_bounds
FROM pg_stats
WHERE tablename IN ('persona', 'recurso', 'seguimiento_acceso')
ORDER BY tablename, attname;
```

d) Filtrado por fecha: plan inicial y mejora.

d.a) Plan inicial (sin índice):

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM seguimiento_acceso
WHERE fecha_hora_entrada = TIMESTAMP '2023-05-06 00:00:00';
```

```
Seq Scan on seguimiento_acceso (cost=...)
  Filter: (fecha_hora_entrada = '2023-05-06 00:00:00'::timestamp)
```

A1 del Teórico 6: recorre todos los bloques y descarta los que no cumplen.

d.b) Mejora: crear índice + ANALYZE.

```
CREATE INDEX IF NOT EXISTS idx_seguimiento_acceso_fecha_entrada
ON seguimiento_acceso (fecha_hora_entrada);

ANALYZE seguimiento_acceso;

EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM seguimiento_acceso
WHERE fecha_hora_entrada = TIMESTAMP '2023-05-06 00:00:00';
```

```
Bitmap Heap Scan on seguimiento_acceso ...
  Recheck Cond: (fecha_hora_entrada = '2023-05-06 00:00:00'::timestamp)
-> Bitmap Index Scan on idx_seguimiento_acceso_fecha_entrada
```

Equivale a A5/A7 del Teórico 6.

e) NATURAL JOIN: plan habitual y forzado de Merge Join.

e.a) Plan habitual — Hash Join:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM seguimiento_acceso NATURAL JOIN persona;
```

```
Hash Join (cost=...)
  Hash Cond: (seguimiento_acceso.persona_id = persona.persona_id)
-> Seq Scan on seguimiento_acceso
-> Hash
    -> Seq Scan on persona
```

e.b) Forzar Merge Join: darle entradas ordenadas por la clave de join.

```
-- 1) Índice ordenado sobre la columna de join (la otra ya tiene PK).
CREATE INDEX IF NOT EXISTS idx_seguimiento_acceso_persona_id
ON seguimiento_acceso (persona_id);
ANALYZE seguimiento_acceso;
ANALYZE persona;
```



```
-- 2) Reescribir la consulta con join explícito y ORDER BY por la clave.
EXPLAIN (ANALYZE, BUFFERS)
SELECT sa.*, p.*
FROM seguimiento_acceso sa
JOIN persona p ON sa.persona_id = p.persona_id
ORDER BY sa.persona_id;
```

```
Merge Join
Merge Cond: (sa.persona_id = p.persona_id)
-> Index Scan using idx_seguimiento_acceso_persona_id on seguimiento_acceso sa
-> Index Scan using persona_pkey on persona p
```

Verificación forzando el algoritmo:

```
SET enable_hashjoin = off;
EXPLAIN (ANALYZE, BUFFERS)
SELECT sa.*, p.*
FROM seguimiento_acceso sa
JOIN persona p ON sa.persona_id = p.persona_id
ORDER BY sa.persona_id;
RESET enable_hashjoin;
```

Por qué funciona / por qué es mejor

- **Sin índice:** el planner sólo tiene Seq Scan; el costo $\approx b_r \cdot t_T$ no cambia con el predicado, así que cualquier filtro lo paga.
- **Con índice + ANALYZE:** el planner conoce $n_distinct$, MCVs e histograma de `fecha_yhora_entrada`; si la selectividad pedida es baja, estima Bitmap Index Scan más barato que Seq Scan y lo elige.
- **Hash Join elegido por defecto en (e):** *hash join* es barato cuando una relación cabe en memoria (Persona) y la otra es grande (seguimiento_acceso); el costo es lineal en ambas. Sin orden previo ni índice utilizable, no hay forma de hacer Merge Join sin un sort costoso.
- **Merge Join con índice + ORDER BY:** si las dos entradas vienen ordenadas (Index Scan sobre `persona_id` en ambas), el merge es lineal y evita la construcción del hash; cambia el cálculo de costo del planner.
- **SET enable_hashjoin = off:** fuerza al planner a descartar Hash Join. Útil pedagógicamente; en producción no se usa: si el planner elige Hash Join es porque, dadas las estadísticas, es lo más barato.

En MySQL 8+

MySQL 8+ tiene equivalentes para todas las operaciones de este ejercicio:

Plan inicial / mejorado (inciso d):

```
-- Plan inicial (sin índice): EXPLAIN ANALYZE existe desde MySQL 8.0.18.
EXPLAIN ANALYZE
SELECT * FROM seguimiento_acceso
WHERE fecha_yhora_entrada = '2023-05-06 00:00:00';
-- type=ALL -> equivale a Seq Scan.

-- Mejora: índice + ANALYZE TABLE
CREATE INDEX idx_seguimiento_acceso_fecha_entrada
ON seguimiento_acceso (fecha_yhora_entrada);
ANALYZE TABLE seguimiento_acceso;

EXPLAIN ANALYZE
```

```
SELECT * FROM seguimiento_acceso
WHERE fecha_hora_entrada = '2023-05-06 00:00:00';
-- type=ref / key=idx_seguimiento_acceso_fecha_entrada -> A5.
```

Forzar el uso de un índice:

```
SELECT * FROM seguimiento_acceso FORCE INDEX (idx_seguimiento_acceso_fecha_entrada)
WHERE fecha_hora_entrada = '2023-05-06 00:00:00';

-- Sugerir (no obligar) un índice:
SELECT * FROM seguimiento_acceso USE INDEX (idx_seguimiento_acceso_fecha_entrada)
WHERE fecha_hora_entrada = '2023-05-06 00:00:00';
```

Hash Join vs Nested Loop (inciso e):

```
-- Plan natural: en MySQL 8.0.18+ el optimizador puede elegir hash join
-- para equi-joins sin índice utilizable; antes solo había nested loop.
EXPLAIN ANALYZE
SELECT * FROM seguimiento_acceso NATURAL JOIN persona;
-- "Inner hash join" en el output indica que se usó hash join.

-- Forzar nested loop (deshabilitar hash join via hint):
SELECT /*+ NO_HASH_JOIN(seguimiento_acceso, persona) */ *
FROM seguimiento_acceso JOIN persona USING (persona_id);

-- Forzar hash join explícito:
SELECT /*+ HASH_JOIN(seguimiento_acceso, persona) */ *
FROM seguimiento_acceso JOIN persona USING (persona_id);
```

`optimizer_trace` — equivalente al `EXPLAIN VERBOSE` de PostgreSQL:

```
SET optimizer_trace = "enabled=on";
SELECT * FROM seguimiento_acceso NATURAL JOIN persona;
SELECT TRACE FROM information_schema.OPTIMIZER_TRACE;
SET optimizer_trace = "enabled=off";
```

Diferencias relevantes con PostgreSQL:

- MySQL no tiene *Merge Join* como algoritmo separado: el resultado equivalente se obtiene combinando Index Scan ordenados con nested loop. PostgreSQL/Oracle sí lo exponen.
- Hash Join en MySQL llegó tarde (8.0.18, oct-2019); en MySQL 5.7 sólo existen Nested Loop y Block Nested Loop. Conviene tener clara la versión del motor al estudiar planes.
- En lugar de `SET enable_hashjoin = off`, MySQL usa hints `/*+ NO_HASH_JOIN(...) */` a nivel sentencia, no a nivel sesión.

Conclusión

El práctico recorre, en orden, los tres niveles donde se puede pensar la ejecución de una consulta:

1. **Algoritmos físicos** (Ejercicio 1): elegir entre A1–A11 en función de qué índices hay y de la forma del predicado.
2. **Reescritura algebraica** (Ejercicios 2 y 3): aplicar reglas de equivalencia para empujar selecciones y proyecciones hacia las hojas y reordenar joins para minimizar tamaños intermedios.
3. **Catálogo y planner reales** (Ejercicios 4 y 5): identificar en PostgreSQL dónde viven las estadísticas que el optimizador usa, y cómo cambia el plan observado actuando sobre ellas

(índices, `ANALYZE`, reescritura con `ORDER BY`, banderas como `enable_hashjoin`).

La idea común detrás de los cinco ejercicios es: *el plan de ejecución no es único*, y la diferencia de costos entre planes equivalentes puede ser de varios órdenes de magnitud; la tarea del DBA y del programador SQL es darle al motor estadísticas y estructuras que le permitan elegir bien. En MySQL 8+, la herramienta análoga a `EXPLAIN (ANALYZE, BUFFERS)` es `EXPLAIN ANALYZE`; el equivalente de las banderas `enable_*` son los hints `/*+ ... */` en la propia sentencia; y los histogramas (que PostgreSQL arma por defecto) en MySQL deben pedirse explícitamente con `ANALYZE TABLE ... UPDATE HISTOGRAM`.